Predicting Acceptance of Personal Loans

UE24CS352A Machine Learning – Mini Project · PES University
Mathin M (PES1UG24AM377) · <Teammate name> (<SRN>)

1. Problem Statement

Thera Bank wants to convert its deposit (liability) customers into personal-loan customers. In the previous campaign only 9.6% of targeted customers accepted. We build a binary classifier that predicts whether a customer will accept a personal loan offer, so the bank can target likely acceptors and reduce campaign cost. We replicate the reference paper's models and then improve on them.

2. Dataset

Kaggle *Personal Loan Modeling* (Bank_Personal_Loan_Modelling.csv): 5000 customers, 14 columns, no missing values or duplicates. Features: Age, Experience, Income, Family, CCAvg (monthly card spend), Education (1–3), Mortgage, and binary flags Securities Account, CD Account, Online, CreditCard. Target **Personal Loan**: 480 yes / 4520 no, so the data is **imbalanced** (a model that always predicts “no” scores 90.4% accuracy). EDA showed acceptors have much higher Income and CCAvg; CD-account holders accept 46% of the time vs 7% otherwise; graduate education and family size 3–4 raise acceptance; Age and Experience are 0.99 correlated and carry little signal.

3. Approach

- **Cleaning:** dropped ID and ZIP Code (identifier / 467-level noise); clipped 52 negative Experience values to 0.
- **Split:** stratified 80/20 train/test (4000 / 1000), positive rate 9.6% in both; StandardScaler fitted on train only.
- **Experiment 1 – Baseline (paper replication):** Perceptron, Logistic Regression, KNN, linear SVM with defaults on unscaled data; K-Means (k=2, clusters mapped to majority label) and MLP (1 hidden layer, 100 neurons) on scaled data.
- **Experiment 2 – Improved:** each model in a Pipeline (Scaler → [SMOTE] → classifier); imbalance handled with class_weight='balanced', XGBoost scale_pos_weight, and SMOTE for KNN/MLP (applied only to training folds); hyper-parameters tuned by GridSearchCV with 5-fold stratified CV on F1; added Random Forest and XGBoost. The final model is chosen on cross-validated F1, never on the test set.
- **Metrics:** precision, recall, F1, ROC-AUC and confusion matrices (accuracy reported but not used for selection).

4. Implementation Overview

Python with pandas, scikit-learn, imbalanced-learn, XGBoost, matplotlib/seaborn. `src/preprocess.py` (load, clean, split, scale), `src/train.py` (both experiments, grid search, saves metrics, figures and `models/best_model.pkl`), `src/evaluate.py` (metrics, ROC, confusion matrices, feature importance), `notebooks/loan_prediction.ipynb` (full EDA and walkthrough) and a Streamlit app `app/app.py` that takes customer details and returns the acceptance probability. Full training runs in about one minute on a laptop.

5. Results (test set: 1000 customers, 96 acceptors)

Experiment	Model	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline	Perceptron	0.911	0.706	0.125	0.212	0.789
Baseline	Logistic Regression	0.952	0.808	0.656	0.724	0.964
Baseline	KNN	0.905	0.508	0.323	0.395	0.875
Baseline	Linear SVM	0.956	0.861	0.646	0.738	0.962
Baseline	K-Means (k=2)	0.904	0.000	0.000	0.000	0.516
Baseline	MLP (100 neurons)	0.987	0.919	0.948	0.933	0.996
Improved	Perceptron	0.877	0.427	0.823	0.562	0.931
Improved	Logistic Regression	0.891	0.466	0.927	0.620	0.963
Improved	KNN + SMOTE	0.959	0.720	0.938	0.815	0.968
Improved	SVM	0.969	0.788	0.927	0.852	0.992
Improved	MLP + SMOTE	0.975	0.838	0.917	0.876	0.994
Improved	Random Forest	0.987	0.903	0.969	0.935	0.999
Improved	**XGBoost**	**0.989**	**0.913**	**0.979**	**0.945**	**0.999**

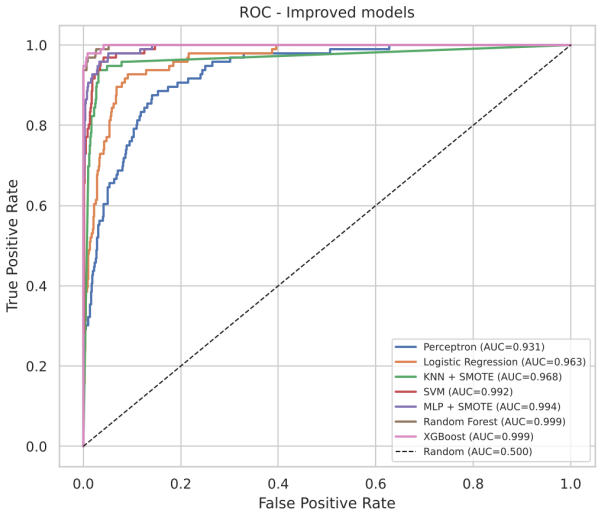


Fig 1. ROC curves of the improved models

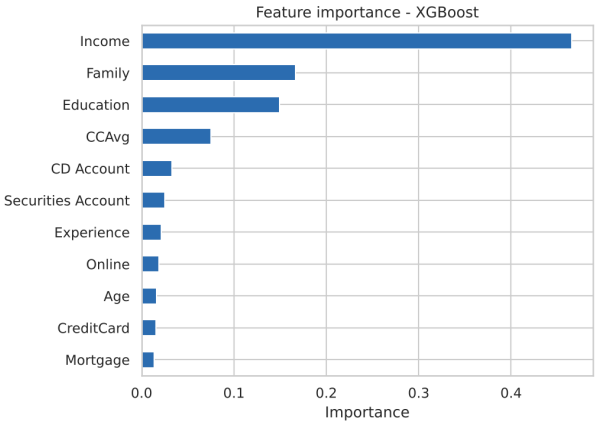


Fig 2. XGBoost feature importance

6. Conclusions

- **XGBoost is the best model**: F1 0.945, recall 0.979, ROC-AUC 0.999. It misses only 2 of 96 acceptors with 9 false positives. Random Forest is a close second (F1 0.935).
- **Income** is by far the strongest predictor, followed by Family, Education and CCAvg; CD Account also helps. Online, CreditCard, Age and Experience contribute little.
- The paper's weak Perceptron / KNN results are largely due to missing feature scaling: scaling plus balancing raised KNN F1 from 0.39 to 0.81 and Perceptron recall from 0.13 to 0.82. K-Means is unsuitable (AUC 0.52) because unsupervised clusters do not align with loan acceptance.
- Class weighting trades precision for recall in linear models (recall > 0.92). For a marketing campaign this is acceptable, since missing a likely customer costs more than an extra call. The non-linear interactions (high income with graduate education and larger family) explain why tree ensembles beat linear models.
- **Future work:** tune the decision threshold to the bank's actual cost trade-off, add SHAP-based explanations, and validate on newer, larger bank data.